COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S3
DATE PERFORMED:	September 17, 2026	**DATE SUBMITTED:**	September 24, 2026
NAME:	Carlo Mari B. Asajar	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 9.1 TREES

1. PROCEDURE OUTPUT

ILO A: Create C++ code to implement a general tree, binary tree and binary search tree.

Task 1: Create code in C++ that will create a tree as shown in the figure above. Use **linked lists** as the internal representation of this tree.

Indicate your code screenshot and comments in the table.
(Table 9-1. General Tree)

Screenshot of Code	**My Code: (GeneralTree.h)** ILOA_B > C GeneralTree.h > ...

```
1   #ifndef GENERALTREE_H
2   #define GENERALTREE_H
3
4   #include <iostream>
5
6   template <typename T>
7   class GeneralTree{
8       private:
9           T key;
10          GeneralTree<T>* firstchild;
11          GeneralTree<T>* sibling;
12
13      public:
14          GeneralTree(T value);
15          void insert(GeneralTree<T>* child);
16          void preOrder();
17          void postOrder();
```

```
18          void inOrder();
19          bool findPreOrder(T findData);
20          bool findPostOrder(T findData);
21          bool findInOrder(T findData);
22          void findData(int choice, T key);
23  };
24
25  // Create a new tree node
26  template <typename T>
27  GeneralTree<T>::GeneralTree(T value){
28      key = value;
29      firstchild = nullptr;
30      sibling = nullptr;
31  }
32
33  // Insert a child node
34  template <typename T>
35  void GeneralTree<T>::insert(GeneralTree<T>* child){
36
```

```
37      // Check if the node has no child
38      if(firstchild == nullptr){
39          firstchild = child;
40      }
41      else{
42
43          // Start with the first child
44          GeneralTree<T>* current = firstchild;
45
46          // Find the last sibling
47          while(current->sibling != nullptr){
48              current = current->sibling;
49          }
50
51          // Add the child as the last sibling
52          current->sibling = child;
53      }
54  }
```

My Code: (main_ILOA_B.cpp)

ILOA_B > main_ILOA_B.cpp > main()

```cpp
#include <iostream>
#include "GeneralTree.h"

int main(){

    // Create the root node
    GeneralTree<char>* A = new GeneralTree<char>('A');

    // Create the child nodes
    GeneralTree<char>* B = new GeneralTree<char>('B');
    GeneralTree<char>* C = new GeneralTree<char>('C');
    GeneralTree<char>* D = new GeneralTree<char>('D');
    GeneralTree<char>* E = new GeneralTree<char>('E');
    GeneralTree<char>* F = new GeneralTree<char>('F');
    GeneralTree<char>* G = new GeneralTree<char>('G');

    // Create the nodes under D and E
    GeneralTree<char>* H = new GeneralTree<char>('H');
    GeneralTree<char>* I = new GeneralTree<char>('I');
    GeneralTree<char>* J = new GeneralTree<char>('J');

    // Create the nodes under F
    GeneralTree<char>* K = new GeneralTree<char>('K');
    GeneralTree<char>* L = new GeneralTree<char>('L');
```

```cpp
    GeneralTree<char>* M = new GeneralTree<char>('M');
    GeneralTree<char>* N = new GeneralTree<char>('N');

    // Create the nodes under J
    GeneralTree<char>* P = new GeneralTree<char>('P');
    GeneralTree<char>* Q = new GeneralTree<char>('Q');

    // Insert the child nodes under A
    A->insert(B);
    A->insert(C);
```

```
35      A->insert(D);
36      A->insert(E);
37      A->insert(F);
38      A->insert(G);
39
40      // Insert H under D
41      D->insert(H);
42
43      // Insert I and J under E
44      E->insert(I);
45      E->insert(J);
46
47      // Insert K, L, and M under F
48      F->insert(K);
49      F->insert(L);
50      F->insert(M);
51
52      // Insert N under G
53      G->insert(N);
54
55      // Insert P and Q under J
56      J->insert(P);
57      J->insert(Q);
```

Observations	The code successfully created the general tree using the ***GeneralTree*** class with the ***firstchild*** and ***sibling*** pointers for the linked-list representation. The ***insert()*** function was used to add each child node and connect the nodes as siblings when more than one child was added under the same parent. In ***main_ILOA_B.cpp***, the nodes from ***A*** to ***Q*** were created and inserted based on the given tree structure, such as ***B, C, D, E, F,*** and ***G*** under **A**, while ***H, I, J, K, L, M, N, P,*** and ***Q*** were placed under their correct parent nodes. The output showed the nodes arranged according to the given general tree, with A as the root and the other nodes connected to their proper parents. Overall, the ***insert()*** function together with the ***firstchild*** and ***sibling*** pointers worked correctly in creating the required tree structure.

Table 9-1. General Tree

Task 2: Complete the following table

Include this as a completed table.
(Table 9-2. Node's Height and Depth)

Node	Height	Depth
A	3	0
B	0	1
C	0	1
D	1	1
E	2	1
F	1	1
G	1	1
H	0	2
I	0	2
J	1	2
K	0	2
L	0	2
M	0	2
N	0	2
P	0	3
Q	0	3

Table 9-2. Node's Height and Depth

ILO B: Create C++ code for implementation of tree traversal methods such as pre-order, in-order and postorder traversal.

Task 3: After implementing the code for above, answer the following:

3.1 Given the tree diagram, find the result of the **pre-order, post-order** and **in-order** traversal strategies by hand.

Include this in the table.
(Table 9-3. Traversal Method Result by Hand)

Pre-order	A, B, C, D, H, E, I, J, P, Q, F, K, L, M, G, N
Post-order	H, Q, P, J, I, M, L, K, N, G, F, E, D, C, B, A
In-order	B, C, H, D, I, P, Q, J, E, K, L, M, F, N, G, A

Table 9-3. Traversal Method Result by Hand

3.2 Create a function for **pre-order, post-order** and **in-order** traversal. Make sure that each function displays an output into the console.

Once you have the output, create and fill the table so that it contains the screenshot of the function, screenshot of the output and your observations. Your observations consist of a comparison between output in **#1** and the output of your functions in **#2**.
(Table 9-4. Traversal Method Functions)

Pre-order Traversal	**My Code: (GeneralTree.h)** 56 // Traverse the tree using Pre-order 57 template <typename T> 58 void GeneralTree<T>::preOrder(){ 59 60 // Visit the current node

```
61      std::cout << key << " ";
62
63      // Traverse the first child
64      if(firstchild != nullptr){
65          firstchild->preOrder();
66      }
67
68      // Traverse the sibling nodes
69      if(sibling != nullptr){
70          sibling->preOrder();
71      }
72  }
```

My Code: (main_ILOA_B.cpp)

```
59      // Display the Pre-order traversal
60      std::cout << "\nPre-order Traversal:" << std::endl;
61      std::cout << "----------------------------------" << std::endl;
62      A->preOrder();
63      std::cout << std::endl;
64
```

My Output:

```
Pre-order Traversal:
----------------------------------
A B C D H E I J P Q F K L M G N
```

Post-order Traversal	**My Code: (GeneralTree.h)** **My Code: (main_ILOA_B.cpp)**

```cpp
// Traverse the tree using Post-order
template <typename T>
void GeneralTree<T>::postOrder(){

    // Traverse the first child
    if(firstchild != nullptr){
        firstchild->postOrder();
    }

    // Traverse the sibling nodes
    if(sibling != nullptr){
        sibling->postOrder();
    }

    // Visit the current node
    std::cout << key << " ";
}
```

```cpp
    // Display the Post-order traversal
    std::cout << "\nPost-order Traversal:" << std::endl;
    std::cout << "---------------------------------" << std::endl;
    A->postOrder();
    std::cout << std::endl;

```

My Output:

```
Post-order Traversal:
---------------------------------
H Q P J I M L K N G F E D C B A
```

In-order Traversal

My Code: (GeneralTree.h)

```
92   // Traverse the tree using In-order
93   template <typename T>
94   void GeneralTree<T>::inOrder(){
95
96       // Traverse the first child
97       if(firstchild != nullptr){
98           firstchild->inOrder();
99       }
100
101      // Visit the current node
102      std::cout << key << " ";
103
104      // Traverse the sibling nodes
105      if(sibling != nullptr){
106          sibling->inOrder();
107      }
108  }
```

	My Code: (main_ILOA_B.cpp) ``` 71 // Display the In-order traversal 72 std::cout << "\nIn-order Traversal:" << std::endl; 73 std::cout << "---------------------------------" << std::endl; 74 A->inOrder(); 75 std::cout << std::endl; 76 ``` **My Output:** ``` In-order Traversal: --------------------------------- B C H D I P Q J E K L M F N G A ```
Observations	The outputs from the functions matched the traversal results found by hand in ***Table 9-3***. The ***preOrder()***, ***postOrder()***, and ***inOrder()*** functions in ***GeneralTree.h*** worked by checking the ***firstchild*** and ***sibling*** pointers and visiting the nodes in their proper order. The ***preOrder()*** output was ***A, B, C, D, H, E, I, J, P, Q, F, K, L, M, G, N,*** where the current node was displayed first before going to the first child and its siblings. The ***postOrder()*** output was ***H, Q, P, J, I, M, L, K, N, G, F, E, D, C, B, A,*** where the child and sibling nodes were visited before displaying the current node. The ***inOrder()*** output was ***B, C, H, D, I, P, Q, J, E, K, L, M, F, N, G, A,*** where the first child was visited before the current node and then the sibling nodes were checked. In ***main_ILOA_B.cpp***, ***A->preOrder(), A->postOrder(),*** and ***A->inOrder()*** were used to display each result in the console. Overall, the three outputs were the same as the manually calculated traversal results, showing that the functions worked correctly and followed the required traversal methods.

Table 9-4. Traversal Method Functions

3.3.Create a function called **findData** that will take the following parameter: **"CHOICE, KEY"**. Choice will determine which traversal method will be used to traverse the tree to find the second parameter **KEY**. If the **KEY** is found, the value will be displayed such that: "**{KEY} was found!**". Otherwise, no output.

Include your output in the table.
(Table 9-5. findData Function)

Screenshot of Code	**My Code: (GeneralTree.h)**

```
110  // Search the tree using Pre-order
111  template <typename T>
112  bool GeneralTree<T>::findPreOrder(T findData){
113
114      // Check the current node
115      if(key == findData){
116          return true;
117      }
118
119      // Search the first child
120      if(firstchild != nullptr){
121          if(firstchild->findPreOrder(findData)){
122              return true;
123          }
124      }
125
126      // Search the sibling nodes
127      if(sibling != nullptr){
128          if(sibling->findPreOrder(findData)){
129              return true;
```

```
130          }
131      }
132
133      return false;
134  }
135
136  // Search the tree using Post-order
137  template <typename T>
138  bool GeneralTree<T>::findPostOrder(T findData){
139
```

```
140     // Search the first child
141     if(firstchild != nullptr){
142         if(firstchild->findPostOrder(findData)){
143             return true;
144         }
145     }
146 
147     // Search the sibling nodes
148     if(sibling != nullptr){
149         if(sibling->findPostOrder(findData)){
150             return true;
151         }
152     }
153 
154     // Check the current node
155     if(key == findData){
156         return true;
157     }
158 
159     return false;
160 }
```

```
161 
162 // Search the tree using In-order
163 template <typename T>
164 bool GeneralTree<T>::findInOrder(T findData){
165 
166     // Search the first child
167     if(firstchild != nullptr){
168         if(firstchild->findInOrder(findData)){
169             return true;
170         }
171     }
172 
173     // Check the current node
174     if(key == findData){
175         return true;
176     }
177 
```

```
178         // Search the sibling nodes
179         if(sibling != nullptr){
180             if(sibling->findInOrder(findData)){
181                 return true;
182             }
183         }
184 
185         return false;
186     }
187 
188     // Find data using the selected traversal
189     template <typename T>
190     void GeneralTree<T>::findData(int choice, T key){
191         bool found = false;
192 
193         // Use Pre-order traversal
194         if(choice == 1){
195             found = findPreOrder(key);
196         }
197 
```

```
198         // Use Post-order traversal
199         else if(choice == 2){
200             found = findPostOrder(key);
201         }
202 
203         // Use In-order traversal
204         else if(choice == 3){
205             found = findInOrder(key);
206         }
207 
208         // Display the result if the key is found
209         if(found){
210             std::cout << key << " was found!" << std::endl;
211         }
212     }
```

My Code: (main_ILOA_B.cpp)

```
77      // Find data using Pre-order traversal
78      std::cout << "\nFind Data - Pre-order:" << std::endl;
79      std::cout << "----------------------------------" << std::endl;
80      A->findData(1, 'J');
81
82      // Find data using Post-order traversal
83      std::cout << "\nFind Data - Post-order:" << std::endl;
84      std::cout << "----------------------------------" << std::endl;
85      A->findData(2, 'M');
86
87      // Find data using In-order traversal
88      std::cout << "\nFind Data - In-order:" << std::endl;
89      std::cout << "----------------------------------" << std::endl;
90      A->findData(3, 'Q');
91
92      // Find data that does not exist
93      std::cout << "\nFind Data - Not Found:" << std::endl;
94      std::cout << "----------------------------------" << std::endl;
95      A->findData(1, 'Z');
96
```

Output

```
Find Data - Pre-order:
----------------------------------
J was found!

Find Data - Post-order:
----------------------------------
M was found!

Find Data - In-order:
----------------------------------
Q was found!
```

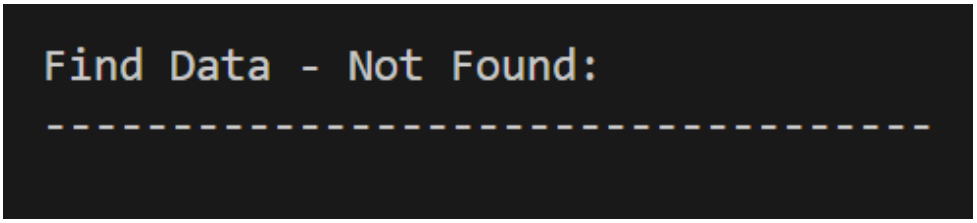
	
Observations	The ***findData()*** function successfully searched for the given key by using the selected traversal method through the ***CHOICE*** and ***KEY*** parameters. In ***GeneralTree.h***, ***choice == 1*** used ***findPreOrder()***, ***choice == 2*** used ***findPostOrder()***, and ***choice == 3*** used ***findInOrder()*** so the program could search the tree based on the selected method. Each search function checked the current node, the ***firstchild***, and the ***sibling*** nodes until the given key was found or all possible nodes were checked. In ***main_ILOA_B.cpp***, ***A->findData(1, 'J')***, ***A->findData(2, 'M')***, and ***A->findData(3, 'Q')*** were used to test the three different search methods, and the output showed that J, M, and Q were found successfully. The ***A->findData(1, 'Z')*** test was also used to check a key that was not included in the tree, and no result was displayed because Z did not exist in the tree. The output followed the required format of ***"{KEY} was found!"*** for the successful searches. Overall, the ***findData()*** function worked correctly because the selected traversal was properly connected to its corresponding search function, and the program was able to identify both existing and non-existing keys based on the given tree structure.

Table 9-5. findData Function

3.4. Add a new leaf node called which will contain the data value of **"O"**. Make this node the child of **nodeG**. Perform the **findData** function you created in **#3**, what can you observe about the output?

Include your output and answers in the table.
(Table 9-6. New Leaf Node)

Screenshot of Code	**My Code: (main_ILOA_B.cpp)** 97 `// Create the new leaf node O` 98 `GeneralTree<char>* O = new GeneralTree<char>('O');` 99 100 `// Insert O under G` 101 `G->insert(O);` 102 103 `// Find the new node O using Pre-order` 104 `std::cout << "\nNew Leaf Node O - Pre-order:" << std::endl;` 105 `std::cout << "-----------------------------------" << std::endl;` 106 `A->findData(1, 'O');` 107 108 `// Find the new node O using Post-order` 109 `std::cout << "\nNew Leaf Node O - Post-order:" << std::endl;` 110 `std::cout << "-----------------------------------" << std::endl;` 111 `A->findData(2, 'O');` 112 113 `// Find the new node O using In-order` 114 `std::cout << "\nNew Leaf Node O - In-order:" << std::endl;` 115 `std::cout << "-----------------------------------" << std::endl;` 116 `A->findData(3, 'O');` 117 `std::cout << std::endl;` 118

Output	`New Leaf Node O - Pre-order:`
	`-----------------------------------`
	`O was found!`

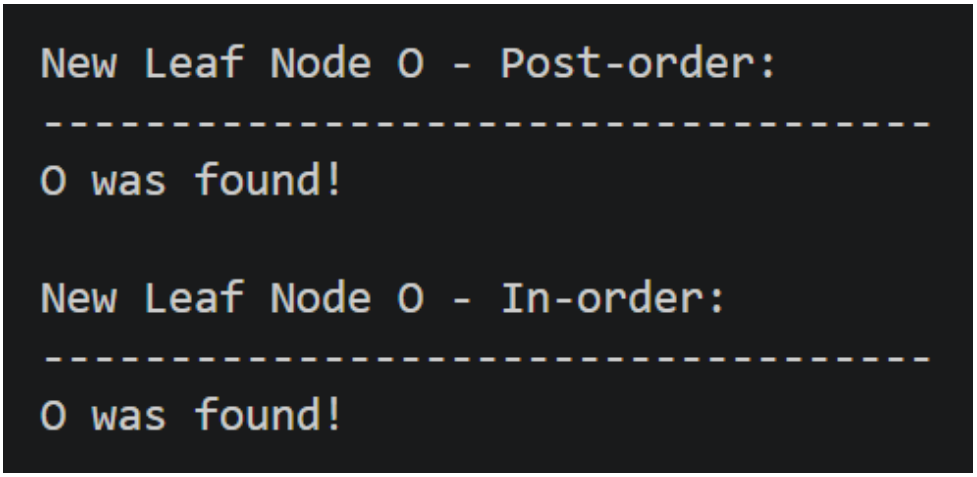
	
Observations	The new leaf node ***O*** was successfully added to the tree by using ***GeneralTree<char>* O = new GeneralTree<char>('O')*** and ***G->insert(O)***, which made ***O*** a child of node ***G*** as required in the task. After adding the new node, the ***findData()*** function was used again with the three different choices to check if ***O*** could be found through each traversal method. The code ***A->findData(1, 'O')*** used Pre-order, ***A->findData(2, 'O')*** used Post-order, and ***A->findData(3, 'O')*** used In-order. The output showed ***O was found!*** for all three searches, which means the new node was properly connected to the tree and could be reached by every search method. Even though each traversal checks the nodes in a different order, all three functions were still able to locate ***O*** because ***G->insert(O)*** placed the new node correctly under ***G***. Since ***O*** is a leaf node, there was no child node below ***O*** that needed to be checked, so the search could continue normally after reaching the new node. Overall, the output showed that adding ***O*** did not cause problems with the existing tree, and the ***findData()*** function continued to work correctly for the newly added node.

Table 9-6. New Leaf Node

My Output: (Complete)

Part 1

```
Pre-order Traversal:
---------------------------------
A B C D H E I J P Q F K L M G N

Post-order Traversal:
---------------------------------
H Q P J I M L K N G F E D C B A

In-order Traversal:
---------------------------------
B C H D I P Q J E K L M F N G A

Find Data - Pre-order:
---------------------------------
J was found!

Find Data - Post-order:
---------------------------------
M was found!
```

Part 2

```
Find Data - In-order:
---------------------------------
Q was found!

Find Data - Not Found:
---------------------------------

New Leaf Node O - Pre-order:
---------------------------------
O was found!

New Leaf Node O - Post-order:
---------------------------------
O was found!

New Leaf Node O - In-order:
---------------------------------
O was found!
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve given problems using the tree data structure's implementation in C++

Step 1: Implement a binary search tree that will take the following values: 2, 3, 9, 18, 0, 1, 4, 5.

My Work:

Screenshot of Code

My Code: (BinarySearchTree.h)

ILOC > C BinarySearchTree.h > ...

```
1   #ifndef BINARYSEARCHTREE_H
2   #define BINARYSEARCHTREE_H
3
4   #include <iostream>
5
6   // Create a Binary Search Tree node
7   template <typename T>
8   class BinarySearchTree{
9       private:
10          T key;
11          BinarySearchTree<T>* left;
12          BinarySearchTree<T>* right;
13
14      public:
15          // Constructor
16          BinarySearchTree(T value);
17
18          // Insert a value into the tree
19          void insert(T value);
20  };
21
22  // Create a new tree node
23  template <typename T>
24  BinarySearchTree<T>::BinarySearchTree(T value){
25      key = value;
26      left = nullptr;
```

```
27      right = nullptr;
28  }
29
30  // Insert a value into the correct position
31  template <typename T>
32  void BinarySearchTree<T>::insert(T value){
33
34      // Go to the left if the value is smaller
35      if (value < key) {
36          if(left == nullptr){
37              left = new BinarySearchTree<T>(value);
38          } else{
39              left->insert(value);
40          }
41      }
```

```
42
43      // Go to the right if the value is larger
44      else if (value > key) {
45          if (right == nullptr){
46              right = new BinarySearchTree<T>(value);
47          } else{
48              right->insert(value);
49          }
50      }
51  }
```

My Code: (main_ILOC.cpp)

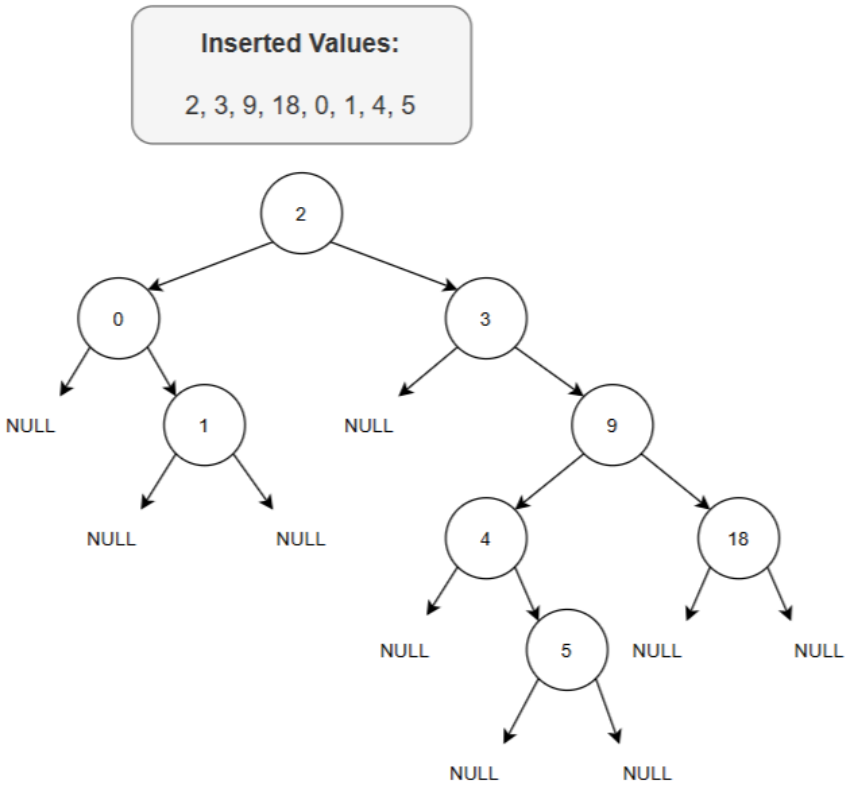
ILOC > main_ILOC.cpp > main()

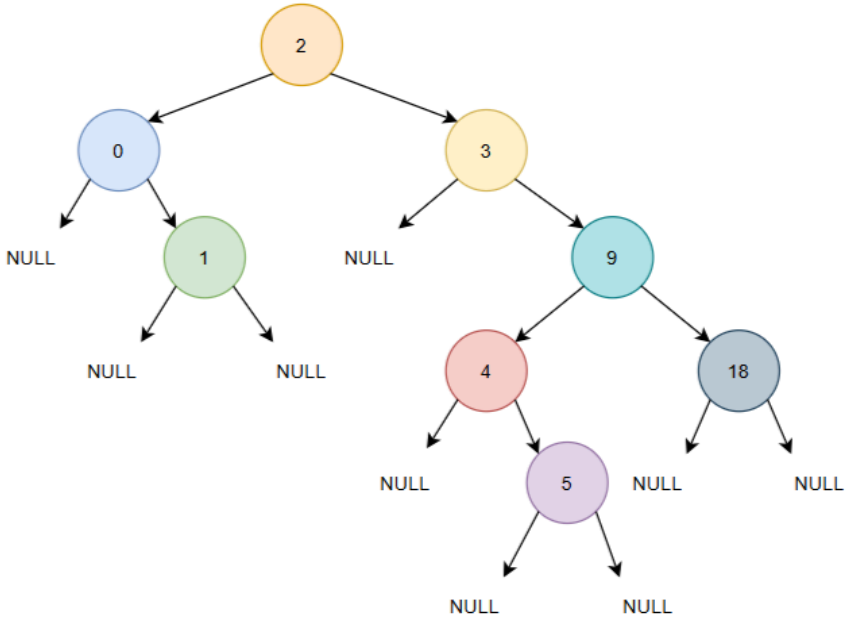
```
1  #include <iostream>
2  #include "BinarySearchTree.h"
3
4  int main(){
5
6      // Create the root node
7      BinarySearchTree<int>* tree = new BinarySearchTree<int>(2);
8
```

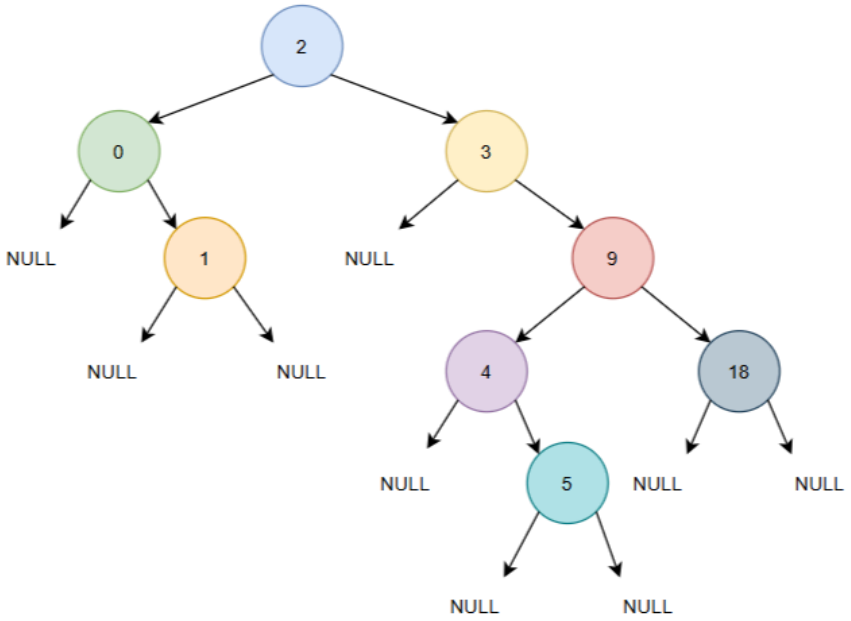
```
9     // Insert values into the tree
10    tree->insert(3);
11    tree->insert(9);
12    tree->insert(18);
13    tree->insert(0);
14    tree->insert(1);
15    tree->insert(4);
16    tree->insert(5);
17
```

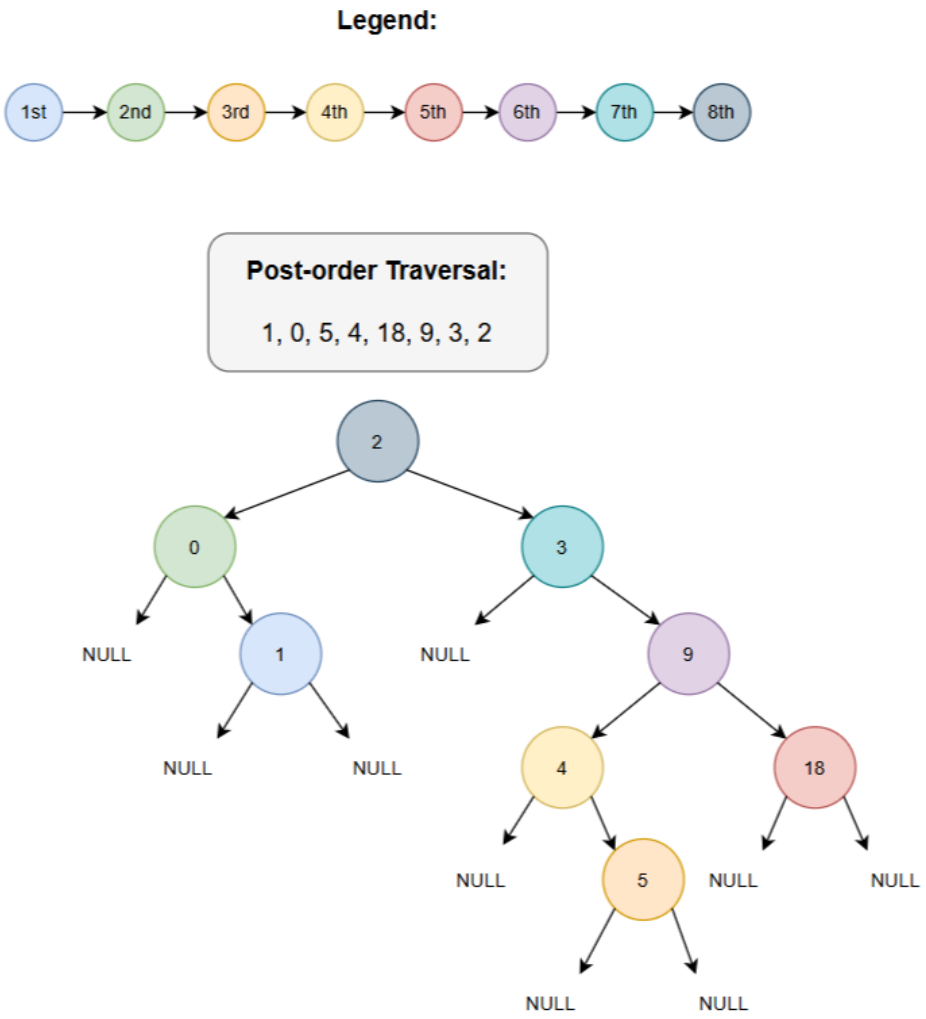
Observation	The binary search tree was successfully created by using the ***BinarySearchTree*** class and the ***insert()*** function to place all the required values in the correct position. In ***main_ILOC.cpp***, ***BinarySearchTree<int>* tree = new BinarySearchTree<int>(2)*** was used to create ***2*** as the root node, followed by ***tree->insert(3)***, ***tree->insert(9)***, ***tree->insert(18)***, ***tree->insert(0)***, ***tree->insert(1)***, ***tree->insert(4)***, and ***tree->insert(5)*** to add the remaining values. In ***BinarySearchTree.h***, the ***insert(T value)*** function checked if the new value was smaller or larger than the current ***key***, where smaller values were placed on the left using ***left*** and larger values were placed on the right using ***right***. The recursive calls ***left->insert(value)*** and ***right->insert(value)*** allowed the values to continue moving through the tree until an empty position was found. The resulting structure followed the Binary Search Tree rule, with values smaller than ***2*** placed on the ***left*** side and larger values placed on the ***right*** side, while the succeeding values were also arranged based on the same comparison. Overall, the code successfully created the required Binary Search Tree and placed all the given values in their proper positions according to the insertion process.

Step 2: Create a diagram to show the tree after all values have been inserted. Then, with the use of visual aids (like arrows and numbers) indicate the traversal order for in-order, pre-order and post-order traversal on the diagram.

My Work:	
Binary Search Tree	

In-order Traversal	**Legend:**  **In-order Traversal:** 0, 1, 2, 3, 4, 5, 9, 18 

Pre-order Traversal	**Legend:**  **Pre-order Traversal:** 2, 0, 1, 3, 9, 4, 5, 18 

Post-order Traversal	
Observation	The tree diagrams successfully showed the final structure after all the values were inserted, and each diagram was different because the arrows and numbered order showed a different way of visiting the nodes. The ***In-order*** diagram followed the ***Left-Root-Right*** order, where the ***left*** side was visited first, followed by the ***root***, then the ***right*** side, giving the output ***0, 1, 2, 3, 4, 5, 9, 18***. The ***Pre-order*** diagram followed the ***Root-Left-Right*** order, where the ***root*** was visited first before going to the ***left*** and ***right*** sides, giving the output ***2, 0, 1, 3, 9, 4, 5, 18***. The ***Post-order*** diagram followed the ***Left-Right-Root*** order, where both the ***left*** and ***right*** sides were visited before going back to the ***root***, giving the output ***1, 0, 5, 4, 18, 9, 3, 2***. The numbered arrows helped differentiate each diagram because the same nodes were present but the visiting numbers and directions changed depending on the traversal method. Overall, the output showed that the three diagrams followed their correct traversal orders while using the same completed Binary Search Tree.

Step 3: Compare the different traversal methods. In-order traversal was performed with what function?

In-order Traversal

My Work:	

Screenshot of Code	**My Code: (BinarySearchTree.h)**
	
	``
	62 // Traverse the tree using In-order
	63 template <typename T>
	64 void BinarySearchTree<T>::inOrder(){
	65
	66 // Visit the left subtree
	67 if(left != nullptr){
	68 left->inOrder();
	69 }
	70
	71 // Display the current node
	72 std::cout << key << " ";
	73
	74 // Visit the right subtree
	75 if(right != nullptr){
	76 right->inOrder();
	77 }
	78 }
	``

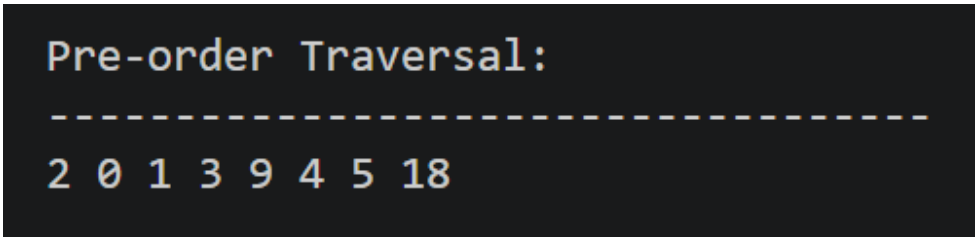
	My Code: (main_ILOC.cpp) ``` 18 // Display the In-order traversal 19 std::cout << "\nIn-order Traversal:" << std::endl; 20 std::cout << "---------------------------------" << std::endl; 21 tree->inOrder(); 22 std::cout << std::endl; 23 ```
Console Output	``` In-order Traversal: --------------------------------- 0 1 2 3 4 5 9 18 ```
Observation	The ***inOrder()*** function successfully performed the In-order traversal by following the ***Left-Root-Right*** order. In ***BinarySearchTree.h***, the function first checked the ***left*** subtree using ***left->inOrder()***, then displayed the current **key**, and after that checked the ***right*** subtree using ***right->inOrder()***. In ***main_ILOC.cpp***, the ***tree->inOrder()*** function call was used to start the traversal and display the result in the console. The output showed ***0 1 2 3 4 5 9 18***, which placed the values from the ***smallest*** to the ***largest*** because the ***left*** subtree was visited first, followed by the ***current*** node, then the ***right*** subtree. The ***recursive*** function continued checking the subtrees until all nodes were visited, so the output matched the expected In-order traversal from the tree diagram. Overall, the ***inOrder()*** function worked correctly and produced the required traversal sequence.

Given the same input values above, what is the output with different traversal methods? For each output below, indicate your observation: is the output different from the pre-order and post-order traversal that you indicated in the diagrams shown in item #2.

Pre-order Traversal

My Work:	

Screenshot of Code	**My Code: (BinarySearchTree.h)**
	80 // Traverse the tree using Pre-order
	81 template <typename T>
	82 void BinarySearchTree<T>::preOrder(){
	83
	84 // Display the current node
	85 std::cout << key << " ";
	86
	87 // Visit the left subtree
	88 if(left != nullptr){
	89 left->preOrder();
	90 }
	91
	92 // Visit the right subtree
	93 if(right != nullptr){
	94 right->preOrder();
	95 }
	96 }

	My Code: (main_ILOC.cpp) ``` 24 // Display the Pre-order traversal 25 std::cout << "\nPre-order Traversal:" << std::endl; 26 std::cout << "----------------------------------" << std::endl; 27 tree->preOrder(); 28 std::cout << std::endl; 29 ```
Console Output	
Observation	The ***preOrder()*** function successfully performed the Pre-order traversal by following the ***Root-Left-Right*** order. In ***BinarySearchTree.h***, the function first displayed the current ***key***, then checked the ***left*** subtree using ***left->preOrder()***, and after that checked the ***right*** subtree using ***right->preOrder()***. In ***main_ILOC.cpp***, the ***tree->preOrder()*** function call was used to start the traversal and display the result in the console. The output showed ***2 0 1 3 9 4 5 18***, which followed the same Pre-order sequence shown in the diagram from ***Step 2*** because the ***root*** was visited first, followed by the ***left*** subtree and then the ***right*** subtree. The ***recursive*** function continued checking the ***left*** and ***right*** subtrees until all nodes were visited, so the output matched the expected Pre-order traversal. Overall, the ***preOrder()*** function worked correctly and produced the required traversal sequence.

Post-order Traversal

My Work:

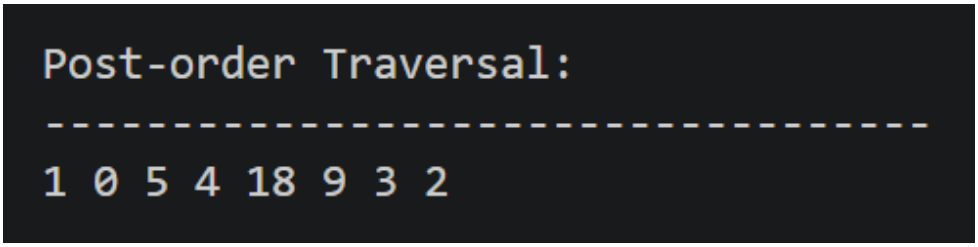
Screenshot of Code

My Code: (BinarySearchTree.h)

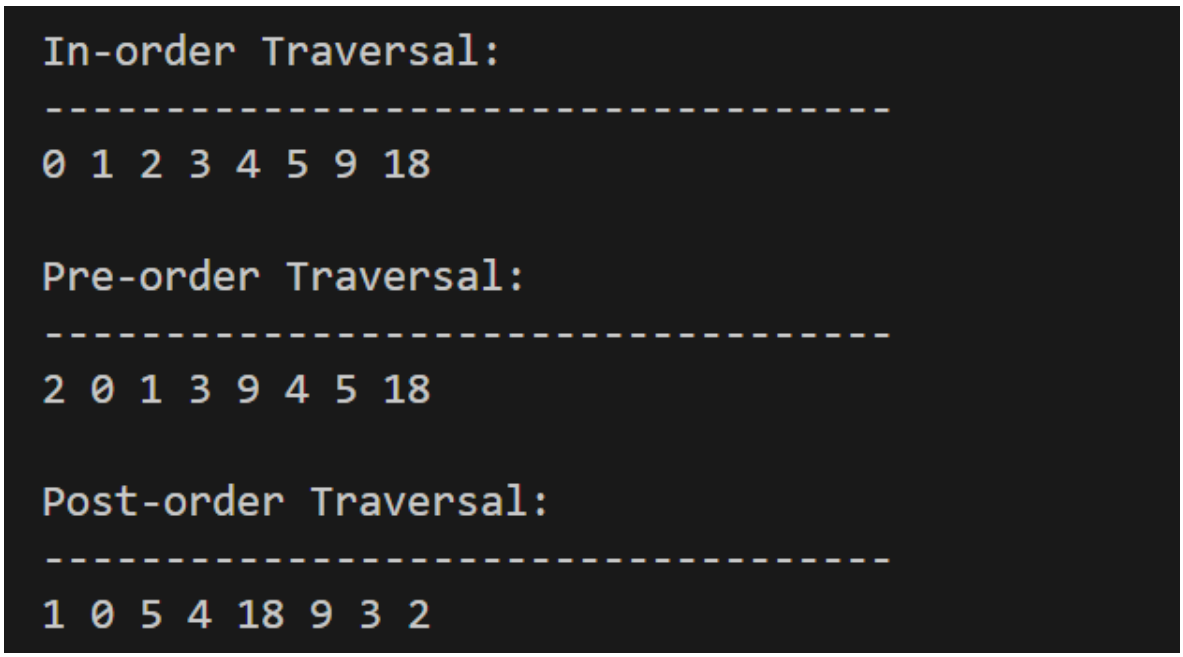
```
98   // Traverse the tree using Post-order
99   template <typename T>
100  void BinarySearchTree<T>::postOrder(){
101
102      // Visit the left subtree
103      if(left != nullptr){
104          left->postOrder();
105      }
106
107      // Visit the right subtree
108      if(right != nullptr){
109          right->postOrder();
110      }
111
112      // Display the current node
113      std::cout << key << " ";
114  }
```

My Code: (main_ILOC.cpp)

```
30      // Display the Post-order traversal
31      std::cout << "\nPost-order Traversal:" << std::endl;
32      std::cout << "----------------------------------" << std::endl;
33      tree->postOrder();
34      std::cout << std::endl << std::endl;
35
```

Console Output	
Observation	The ***postOrder()*** function successfully performed the Post-order traversal by following the ***Left-Right-Root*** order. In ***BinarySearchTree.h***, the function first checked the ***left*** subtree using ***left->postOrder()***, then checked the ***right*** subtree using ***right->postOrder()***, and after that displayed the current ***key***. In ***main_ILOC.cpp***, the ***tree->postOrder()*** function call was used to start the traversal and display the result in the console. The output showed ***1 0 5 4 18 9 3 2***, which followed the same Post-order sequence shown in the diagram from ***Step 2*** because the ***left*** and ***right*** subtrees were visited first before going back to the ***root***. The ***recursive*** function continued checking the subtrees until all nodes were visited, so the output matched the expected Post-order traversal. Overall, the ***postOrder()*** function worked correctly and produced the required traversal sequence.

My Output: (Complete)



Conclusion:

The activity helped in understanding how tree structures work and how they can be created and used through C++. For the lessons learned, General Trees use firstchild and sibling pointers to connect the nodes, while Binary Search Trees use left and right pointers to arrange smaller and larger values. The activity also showed how preOrder(), postOrder(), and inOrder() visit the nodes in different orders, while findData() can search for a specific key using different traversal methods. For the procedure, the codes were created and tested step by step, starting with creating the nodes and connecting them, then applying the different traversal and search functions. The important parts such as insert(), preOrder(), postOrder(), inOrder(), and findData() helped show how each tree operation works. For the supplementary activity, a new leaf node O was added under node G and was searched using the three different traversal methods, while the Binary Search Tree was also tested using the required traversal functions.

One difficulty faced was understanding how the firstchild and sibling pointers move through the General Tree, especially when there are many nodes connected under the same parent. Another difficulty was following the recursive process in the traversal functions because the nodes are visited in different orders depending on the method used. There was also some confusion when checking the traversal outputs against the diagrams, especially when tracing the nodes manually. Even with these difficulties, the results were correct and the outputs matched the expected answers. Overall, the activity was done well because the tree structures and required functions were successfully implemented and tested, but there are still areas for improvement in understanding recursion, tracing tree connections, and identifying traversal orders faster without depending too much on the diagrams or examples. More practice with different tree structures and traversal problems can help improve the understanding and confidence in using these functions.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **formatting, explanation of concepts, grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The AI was used to explain how the code works by giving a detailed explanation on the different parts of the code. Moreover, the AI was used to assist in improving the grammar of the code analysis, allowing the explanation to be written more clearly and fluently in English.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.